

Explain the concept of superposition in quantum computing and its significance for Qubits.

1. What is Superposition?

Superposition is a fundamental principle of quantum mechanics which states that a quantum system can exist in multiple states at the same time. For a qubit, this means it can be in a combination of $|0\rangle$ and $|1\rangle$ simultaneously.

Mathematical Representation :

$$|\Psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where,

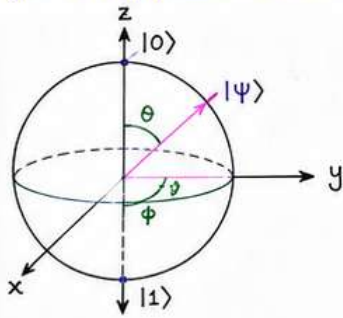
α, β are complex probability amplitudes

$$|\alpha|^2 + |\beta|^2 = 1 \quad (\text{Normalization condition})$$

$|0\rangle$ and $|1\rangle$ are the computational basis states.

2. Visualizing Superposition

A qubit in superposition can be represented on the Bloch sphere.



$$|\Psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle$$

$$0 \leq \theta \leq \pi, \quad 0 \leq \phi < 2\pi$$

Every point on the surface represents a unique qubit state (a superposition).

3. Significance of Superposition for Qubits

- **Enables Quantum Parallelism** : A qubit can represent both $|0\rangle$ and $|1\rangle$ at the same time. With n qubits, 2^n basis states are processed simultaneously.
- **Exponential State Space** : n qubits can exist in a superposition of 2^n states $\rightarrow$ huge computational advantage.
- **Foundation for Quantum Algorithms** : Algorithms like Deutsch-Jozsa, Grover's search, and Shor's factoring exploit superposition to outperform classical algorithms.
- **Interference Possibility** : Superposed states can interfere constructively or destructively, enhancing the probability of correct outcomes.
- **Resource Efficiency** : Solves certain problems using fewer resources (time, memory) compared to classical systems.

4. Measurement and Collapse

- When a measurement is made, the qubit collapses to either $|0\rangle$ or $|1\rangle$ with probabilities :

$$P(0) = |\alpha|^2, \quad P(1) = |\beta|^2$$

- The original superposition is destroyed after measurement.
- This probabilistic nature is central to quantum computing.

5. Example

If a qubit is in state $|\Psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$

$$\text{Then, } P(0) = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2}, \quad P(1) = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2}$$

Each measurement gives 0 or 1 with 50% probability.

6. Superposition vs Classical Bit

Feature	Classical Bit	Qubit (Superposition)
States	0 or 1	$\alpha 0\rangle + \beta 1\rangle$
Information	1 bit	Multiple states at once
System of n bits	Represents 1 state	Represents 2^n states
Computation	Sequential	Quantum parallelism
Measurement	Deterministic	Probabilistic (collapse)

7. Key Takeaway

Superposition allows qubits to hold and process a vast amount of information simultaneously. It is the core reason why quantum computers have the potential to solve certain problems much faster than classical computers.

Summary : Superposition is the ability of a qubit to be in a combination of $|0\rangle$ and $|1\rangle$ at the same time, described by $\alpha|0\rangle + \beta|1\rangle$ where $|\alpha|^2 + |\beta|^2 = 1$. It enables quantum parallelism, exponential state space, interference, and forms the basis for the power of quantum algorithms.

Describe the Pauli-X, Y and Z gates and its role in quantum computing.

1. Introduction

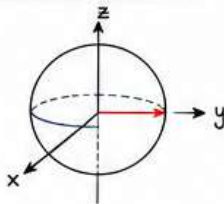
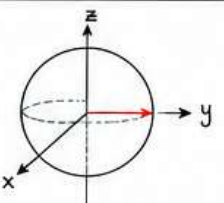
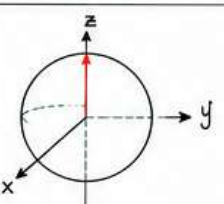
Pauli gates are a set of single-qubit quantum gates corresponding to the Pauli matrices. They are fundamental building blocks for quantum circuits, error correction and many quantum algorithms.

The Pauli Matrices

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

These matrices are Hermitian (self-inverse), unitary and have eigenvalues ± 1 .

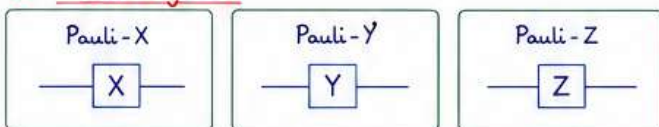
2. Pauli-X, Y and Z Gates

Gate	Matrix	Action on Basis States	Bloch Sphere Effect	Description / Role
Pauli-X (Bit-flip gate)	$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	$X 0\rangle = 1\rangle$ $X 1\rangle = 0\rangle$		- Flips the qubit state between $0\rangle$ and $1\rangle$. - Represents a rotation of π (180°) around the X-axis. - Used in state manipulation and as a basic building block in circuits.
Pauli-Y (Bit-phase flip gate)	$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$	$Y 0\rangle = i 1\rangle$ $Y 1\rangle = -i 0\rangle$		- Flips the state and adds a phase of $\pm i$. - Represents a rotation of π around the Y-axis. - Important in creating arbitrary single-qubit rotations and in quantum algorithms.
Pauli-Z (Phase-flip gate)	$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	$Z 0\rangle = 0\rangle$ $Z 1\rangle = - 1\rangle$		- Leaves $0\rangle$ unchanged and adds a phase of -1 to $1\rangle$. - Represents a rotation of π around the Z-axis. - Crucial in phase encoding and quantum error correction.

3. Key Properties

- Unitary and Hermitian: $X^\dagger = X$, $Y^\dagger = Y$, $Z^\dagger = Z$
- Involutory: $X^2 = Y^2 = Z^2 = I$
- Determinant: $\det(X) = \det(Y) = \det(Z) = -1$
- Anti-commutation: $\{X, Y\} = \{Y, Z\} = \{Z, X\} = 0$
- Commutation: $[X, Z] = [Y, Z] = [X, Y] \neq 0$
(Here $\{A, B\} = AB + BA$ and $[A, B] = AB - BA$)

5. Circuit Symbols



6. Example

Let $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$

- $X|\psi\rangle = \alpha|1\rangle + \beta|0\rangle$
- $Y|\psi\rangle = i\alpha|1\rangle - i\beta|0\rangle$
- $Z|\psi\rangle = \alpha|0\rangle - \beta|1\rangle$

Matrix Relation

$$XYZ = iI$$

$$XY = iZ, \quad YZ = iX, \quad ZX = iY$$

4. Role in Quantum Computing

- Universal Building Blocks: Any single-qubit operation can be constructed using combinations of Pauli gates and phase gates.
- Quantum State Manipulation: They allow flipping and phasing of qubits, enabling control over quantum states.
- Error Detection & Correction: Pauli errors (X, Y, Z) are used to model and correct errors in quantum error-correcting codes (e.g., Shor, Steane codes).
- Measurements: Z-basis measurement is the standard computational basis measurement.
- Stabilizers in QEC: Stabilizer codes use tensor products of Pauli operators as stabilizers.

Summary: Pauli-X, Y and Z gates correspond to bit flip, bit-phase flip and phase flip operations respectively. They are fundamental single-qubit gates used for state manipulation, building complex circuits, and ensuring reliable quantum computation through error correction and stabilization.

Explain the concept of quantum parallelism and how it is utilized in Deutsch's algorithm.

1. Quantum Parallelism

Quantum parallelism is the ability of a quantum computer to evaluate a function on many inputs simultaneously using superposition.

If we have n qubits, they can be in a superposition of all 2^n possible states at once:

$$|\Psi\rangle = \sum_{x=0}^{2^n-1} \alpha_x |x\rangle$$

Thus a single quantum operation can process all possible inputs in parallel.

It is not "doing many things at once" in the classical sense, but rather exploiting interference to obtain the correct answer with high probability.

2. Deutsch Problem

Given a black-box function $f: \{0,1\} \rightarrow \{0,1\}$, determine whether f is:

- Constant: $f(0) = f(1)$
- Balanced: $f(0) \neq f(1)$

Classically, we may need two evaluations of f .

Deutsch's algorithm solves it with only one evaluation.

3. Deutsch's Algorithm (Overview)

- ① Start with two qubits initialized to $|0\rangle |1\rangle$.
- ② Apply Hadamard gates to both qubits to create superposition.
- ③ Apply the oracle U_f which computes:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

This evaluates $f(x)$ for $x=0$ and 1 simultaneously.

- ④ Apply Hadamard gate to the first qubit.
- ⑤ Measure the first qubit.
If the result is $|0\rangle \rightarrow f$ is constant
If the result is $|1\rangle \rightarrow f$ is balanced
(The second qubit is not measured.)

4. How Quantum Parallelism is Utilized

After step ②, the first qubit is in superposition:

$$\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) |1\rangle$$

superposition of all inputs (0 and 1) ancilla qubit prepared in $|1\rangle$

When U_f is applied, the function f is evaluated on both inputs at the same time:

$$\frac{1}{\sqrt{2}} (|0\rangle |1 \oplus f(0)\rangle + |1\rangle |1 \oplus f(1)\rangle)$$

Interference created in the next Hadamard gate on the first qubit makes the two cases (constant or balanced) give orthogonal results.

The measurement of the first qubit reveals the nature of f with certainty.

Step-by-Step State Evolution

Step	Operation	State of the system (first qubit, second qubit)
1	Initialize	$ 0\rangle 1\rangle$
2	$H \otimes H$	$\frac{1}{2} (0\rangle + 1\rangle) (0\rangle - 1\rangle)$ $= \frac{1}{\sqrt{2}} (0\rangle - 1\rangle)_{\text{first}} \otimes -\rangle$
3	U_f	$\frac{1}{\sqrt{2}} [0\rangle f(0)\rangle - 1\rangle f(1)\rangle]$
4	H on first qubit	$\frac{1}{2} [(0\rangle (f(0)\rangle - f(1)\rangle) + (1\rangle (f(0)\rangle + f(1)\rangle))]$
5	Measure first qubit	• If f is constant ($f(0) = f(1)$) $\rightarrow 0\rangle$ • If f is balanced ($f(0) \neq f(1)$) $\rightarrow 1\rangle$

Example

- If $f(x) = 0$ (constant) $\rightarrow$ Outcome = $|0\rangle$
- If $f(x) = x$ (balanced) $\rightarrow$ Outcome = $|1\rangle$

Summary: Quantum parallelism allows a quantum computer to evaluate a function on all possible inputs simultaneously using superposition. In Deutsch's algorithm, this enables the computer to determine whether a function is constant or balanced with just one evaluation of the function, whereas a classical computer needs two.

What is the main challenge in classical factoring algorithms, and how does Shor's algorithm overcome it?

1. Factoring Problem

Given a large composite number N , find its non-trivial factors (a and b) such that $N = a \times b$ and $1 < a, b < N$. This forms the basis of many cryptographic systems like RSA.

2. Main Challenge in Classical Algorithms

- The best known classical algorithms (e.g., General Number Field Sieve - GNFS) have sub-exponential time complexity:

$$L_N \left[\frac{1}{3}, \left(\frac{64}{9} \right)^{1/3} \right] = \exp \left(\left(\frac{64}{9} \right)^{1/3} (\ln N)^{2/3} (\ln \ln N)^{1/3} \right)$$

- This is still super-polynomial, meaning the time required grows extremely fast as N increases.
- For sufficiently large N (e.g., 2048-bit numbers), classical factoring becomes computationally infeasible.

Hence, the main challenge is the exponential (or sub-exponential) time complexity of classical factoring algorithms on large numbers.

3. Key Idea of Shor's Algorithm

Shor's algorithm reduces factoring to the problem of finding the period (r) of a function:

$$f(x) = a^x \bmod N$$

If we can find r (the smallest positive integer such that $a^r \equiv 1 \pmod{N}$), then with high probability:

- If r is even:

$$p = \gcd(a^{r/2} - 1, N)$$

$$q = \gcd(a^{r/2} + 1, N)$$

are non-trivial factors of N .

- If r is odd or $\gcd(\dots)$ gives trivial factors, choose another a and repeat.

Thus, factoring is solved efficiently if we can find the period r of $f(x)$.

4. How Shor's Algorithm Overcomes the Challenge

Shor's algorithm finds the period r exponentially faster using quantum computation.

Steps:

- Randomly choose a number a such that $1 < a < N$ and $\gcd(a, N) = 1$.
- Use a quantum computer to find the period r of $f(x) = a^x \bmod N$.
- If r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, compute $p = \gcd(a^{r/2} - 1, N)$ and $q = \gcd(a^{r/2} + 1, N)$.
- If p and q are non-trivial factors, stop. Otherwise, repeat with another a .

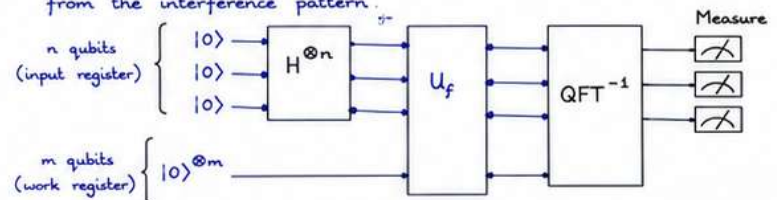
Quantum Advantage:

The period finding can be done in polynomial time $O((\log N)^3)$ on a quantum computer, whereas no known classical algorithm can do this efficiently.

5. Quantum Period Finding (Core of Shor's Algorithm)

Shor's algorithm uses:

- Superposition to evaluate $f(x)$ for many x values simultaneously.
- Quantum Fourier Transform (QFT) to extract the period r from the interference pattern.



U_f performs: $|x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle$

QFT^{-1} extracts the period r from the phase information.

6. Complexity Comparison

Algorithm	Time Complexity	Remarks
Best Classical (GNFS)	Sub-exponential $L_N \left[\frac{1}{3}, \left(\frac{64}{9} \right)^{1/3} \right]$	Impractical for large N
Shor's Algorithm	Polynomial $O((\log N)^3)$	Efficient on a quantum computer

Summary: The main challenge in classical factoring is the super-polynomial (sub-exponential) time complexity, making it infeasible for large numbers. Shor's algorithm overcomes this by reducing factoring to the period finding problem and solving it in polynomial time using quantum parallelism and the Quantum Fourier Transform.

What is quantum annealing and how it relates to finding the ground state of a system.

1. Quantum Annealing - Definition

Quantum annealing (QA) is a metaheuristic optimization technique that uses quantum fluctuations to find the minimum (ground state) of a given objective function. It is especially useful for solving combinatorial optimization problems.

2. Basic Idea

Any optimization problem can be mapped to finding the state of a physical system with the lowest possible energy. Quantum annealing prepares a system in an easy-to-prepare ground state of a simple Hamiltonian, then slowly changes (anneals) it into a Hamiltonian whose ground state encodes the solution to the problem.

3. The Hamiltonian and Annealing Process

We define a time-dependent Hamiltonian:

$$H(t) = A(t)H_D + B(t)H_P, \quad 0 \leq t \leq T$$

- H_D - Driver (initial) Hamiltonian with an easy ground state.
- H_P - Problem Hamiltonian whose ground state encodes the solution.
- $A(t), B(t)$ - Annealing schedules:
 $A(0) \gg B(0)$ (start with H_D)
 $A(T) \approx 0, B(T) \gg 0$ (end with H_P)

The system is evolved slowly from $t=0$ to T .

4. Adiabatic Theorem (Key Principle)

If the evolution is slow enough and the energy gap $\Delta(t)$ between the ground state and first excited state remains large, the system stays in its instantaneous ground state.

Therefore, at $t=T$, the system is in (or very close to) the ground state of H_P .

5. Relation to Ground State

The ground state of the final Hamiltonian H_P corresponds to the optimal solution (minimum energy) of the problem. Measuring the system at the end gives the bit string (or state) that represents this optimal solution.

6. Example: Mapping to Ising Model

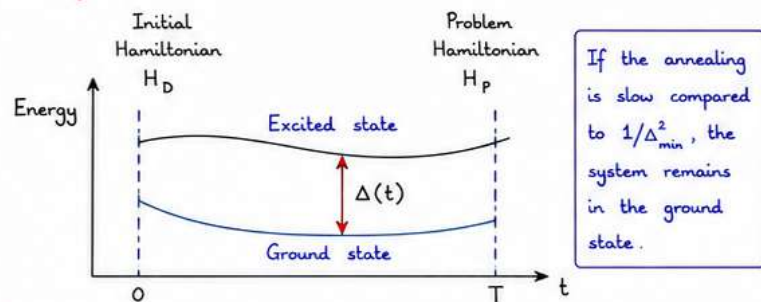
Many problems (e.g., Max-Cut, SAT, TSP) can be mapped to an Ising Hamiltonian:

$$H_P = \sum_i h_i \sigma_i^z + \sum_{i < j} J_{ij} \sigma_i^z \sigma_j^z$$

where $\sigma_i^z = \pm 1$ represents spin (qubit) states.

The ground state (minimum energy) of H_P gives the optimal answer.

7. Physical Picture



8. Applications

- Combinatorial optimization (e.g., Max-Cut, Portfolio optimization)
- Machine learning (feature selection, clustering)
- Scheduling, routing, and constraint satisfaction problems
- Drug discovery, material design, and many more

9. Key Points

- Quantum annealing uses quantum fluctuations (tunneling) to escape local minima.
- It does not guarantee the global optimum but can find good solutions faster for some hard problems.
- Implemented in devices like D-Wave quantum annealers.

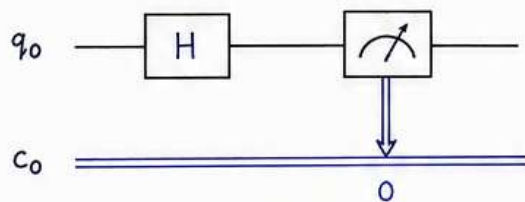
Summary: Quantum annealing is an optimization method that finds the ground state of a problem Hamiltonian by slowly evolving a quantum system from an easy initial Hamiltonian to the problem Hamiltonian. According to the adiabatic theorem, if the evolution is slow enough, the system remains in its ground state, giving the optimal (minimum energy) solution to the problem.

Q. Write the Qiskit code to create a simple quantum circuit that places a single qubit in an equal superposition state and then measures it.

Answer :

A single qubit can be put into an equal superposition state of $|0\rangle$ and $|1\rangle$ by applying a Hadamard gate (H) on it. The Hadamard gate transforms the basis state $|0\rangle$ to $(|0\rangle + |1\rangle)/\sqrt{2}$. Measuring this qubit yields '0' or '1' with approximately 50% probability each.

Quantum Circuit :



H $\rightarrow$ Hadamard gate (creates equal superposition)

Measurement $\rightarrow$ Collapses the qubit to '0' or '1' and stores the result in classical bit c_0 .

Qiskit Code :

```
from qiskit import QuantumCircuit, Aer, execute      # Import required modules
from qiskit.visualization import plot_histogram

# 1. Create a quantum circuit with 1 qubit and 1 classical bit
qc = QuantumCircuit(1, 1)                            # 1 qubit, 1 classical bit

# 2. Put the qubit (q0) in equal superposition using Hadamard gate
qc.h(0)                                                # Apply H gate on qubit 0

# 3. Measure the qubit
qc.measure(0, 0)                                       # Measure qubit 0 and store result in c0

# 4. Choose simulator and execute the circuit
simulator = Aer.get_backend('qasm_simulator')        # Use QASM simulator
job = execute(qc, simulator, shots=1024)              # Run the circuit for 1024 shots
result = job.result()                                  # Get the result
counts = result.get_counts(qc)                        # Get the counts of outcomes

# 5. Display the result
print("Counts:", counts)                              # Print the counts
plot_histogram(counts)                                # Plot histogram
```

Explanation :

- $qc.h(0)$ puts the qubit in state $(|0\rangle + |1\rangle)/\sqrt{2}$ (equal superposition).
- After measurement, the qubit collapses to either $|0\rangle$ or $|1\rangle$.
- Running the circuit many times (shots) gives '0' and '1' outcomes with approx. equal probability.

Expected Output :

On running the above code, the output will be similar to:

Counts: {'0': 512, '1': 512} (values may vary slightly each run)

The histogram will show two bars of nearly equal height for '0' and '1'.

Implementation of Toffoli Gate using Qiskit

The Toffoli gate (CCX gate) is a 3-qubit controlled-controlled-NOT gate. It flips the target qubit (t) only when both control qubits (c_1 and c_2) are 1.

1. Toffoli Gate (CCX Gate) :

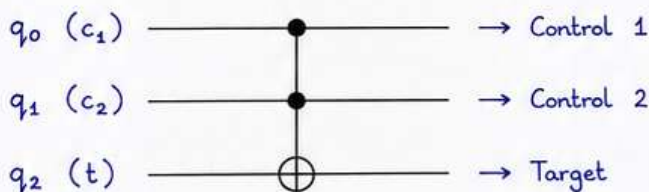
It is a universal reversible gate. It leaves the target qubit unchanged if either or both control qubits are 0. It flips the target qubit when $c_1 = 1$ and $c_2 = 1$.

2. Truth Table :

c_1	c_2	t (in)	t (out)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Target flips
($t_{out} = t_{in} \oplus 1$)
only when
 $c_1 = 1$ and $c_2 = 1$

3. Quantum Circuit for Toffoli Gate :



q_2 flips (X operation) only when q_0 and q_1 are 1.

4. Qiskit Implementation :

```
from qiskit import QuantumCircuit, Aer, execute      # Import required modules
from qiskit.visualization import plot_histogram

# Create a quantum circuit with 3 qubits and 3 classical bits
qc = QuantumCircuit(3, 3)      # q0, q1 → controls ; q2 → target

# Apply Toffoli gate (ccx)
qc.ccx(0, 1, 2)                # Toffoli gate with controls q0, q1
                                # and target q2

# Measure all qubits
qc.measure([0, 1, 2], [0, 1, 2]) # Map q0 → c0, q1 → c1, q2 → c2

# Choose simulator and run the circuit
simulator = Aer.get_backend('qasm_simulator')      # QASM simulator
job = execute(qc, simulator, shots=1024)           # Execute with 1024 shots
result = job.result()                               # Get result
counts = result.get_counts(qc)                     # Get counts of outcomes

# Display results
print("Counts : ", counts)                          # Print counts
plot_histogram(counts)                              # Plot histogram
```

5. Explanation :

- $qc.ccx(0, 1, 2)$ applies a Toffoli gate with q_0 and q_1 as control qubits and q_2 as target qubit.
- The target qubit q_2 flips (X gate) only when $q_0 = 1$ and $q_1 = 1$.
- All qubits are measured and results are stored in classical bits c_0, c_1, c_2 respectively.

6. Expected Output :

Since we start from $|000\rangle$, after Toffoli gate the state remains $|000\rangle$ (target does not flip because controls are not both 1).

Counts : {'000': 1024}

Histogram : (one bar at 000)

Note : If the input state were $|110\rangle$ or $|111\rangle$, the target q_2 would flip. For example, starting from $|110\rangle \rightarrow$ Toffoli gate $\rightarrow |111\rangle$.

Outline the steps to set up an IBM Quantum Experience account

IBM Quantum Experience is a free platform that allows you to design, simulate and run quantum circuits on real quantum hardware.

① Step 1 : Go to the IBM Quantum website

- Open your web browser and visit the IBM Quantum platform:
<https://quantum.ibm.com/>

② Step 2 : Sign in or Create an IBM Account

- Click on "Sign in" at the top-right corner.
- If you already have an IBM account, enter your email and password and sign in.
- If you don't have an account, click on "Create an IBM Account".
- Fill in the required information such as your email address and name.
- Choose a password and accept the IBM Terms of Use.

③ Step 3 : Verify Your Email Address

- After registration, IBM will send a verification email to your inbox.
- Open the email and click on the verification link to confirm your email address.
- Once verified, you can sign in to your IBM Quantum account.

④ Step 4 : Complete Your Profile (Optional but Recommended)

- After signing in, you may be asked to complete your profile.
- Provide additional details such as your name, organization (optional), country, and how you plan to use IBM Quantum.
- Click "Save" or "Complete Profile".

⑤ Step 5 : Access the IBM Quantum Platform

- Once logged in, you will be redirected to the IBM Quantum Dashboard.
- You can now access the Composer (to build and simulate circuits), backends (real quantum systems), and documentation.

⑥ Step 6 : (Optional) Link to GitHub for Advanced Features

- You can link your IBM Quantum account to GitHub for saving projects, sharing notebooks, and using the IBM Quantum API.
- Go to "Account" settings → "Linked Accounts" → "GitHub" and follow the steps.

⑦ Step 7 : Start Using IBM Quantum

- You have successfully created your IBM Quantum Experience account.
- You can now design quantum circuits, run them on simulators or real hardware, and explore quantum computing!

Note : The IBM Quantum platform offers a free plan that allows you to run jobs on quantum hardware with certain usage limits. You can upgrade to a paid plan for higher usage and additional features.